
Geospatial file formats and packages

Rennan Marujo
Édipo Cremon





Jupyter



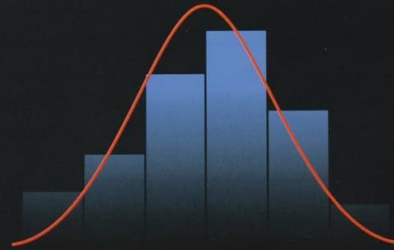
—
R

Environment

R

Marcio Pupin Mello
Luiz Alexandre Peternelli

Conhecendo o R



Uma visão mais que
Estatística

EDITORA
UFV

Springer Texts in Statistics

Gareth James
Daniela Witten
Trevor Hastie
Robert Tibshirani

An Introduction to Statistical Learning

with Applications in R
Second Edition

Springer

Springer Texts in Statistics

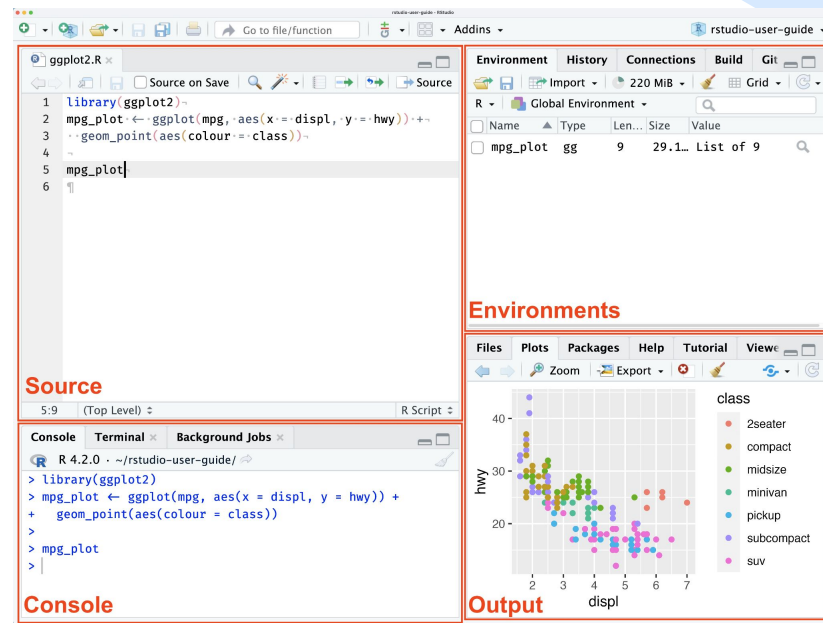
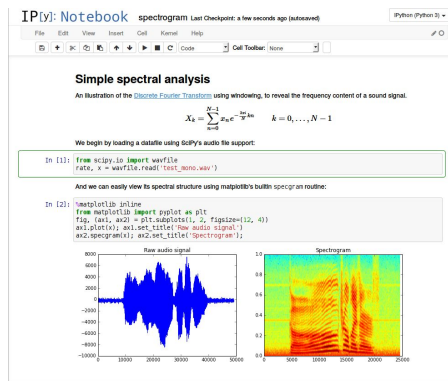
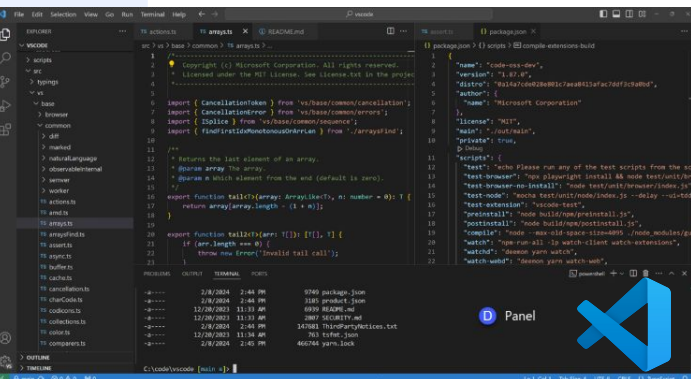
Gareth James · Daniela Witten · Trevor Hastie ·
Robert Tibshirani · Jonathan Taylor

An Introduction to Statistical Learning

with Applications in Python

Springer

Integrated Development Environment (IDE)





Python Environment

Conda

```
wget https://repo.anaconda.com/archive/Anaconda3-2025.12-2-Linux-x86\_64.sh  
chmod +x Anaconda3-2025.12-2-Linux-x86\_64.sh  
./Anaconda3-2025.12-2-Linux-x86\_64.sh
```

yes

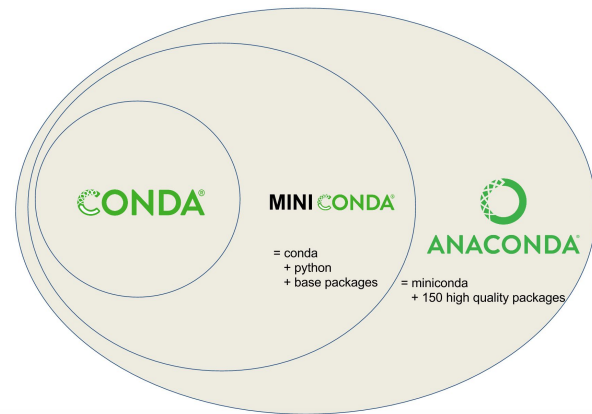
Enter

yes

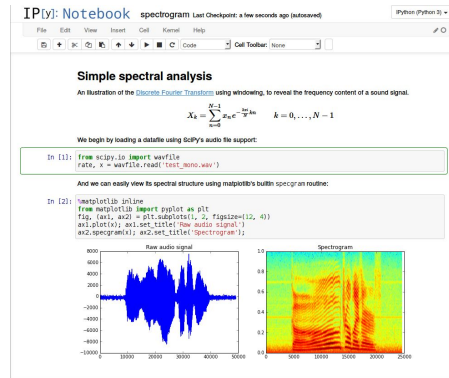
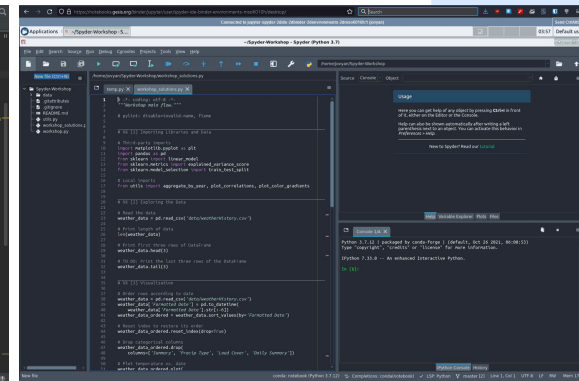
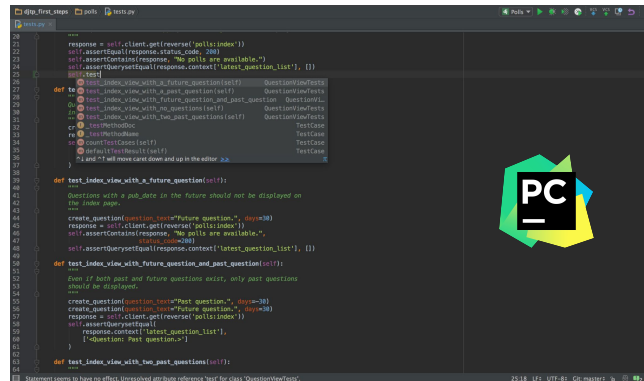
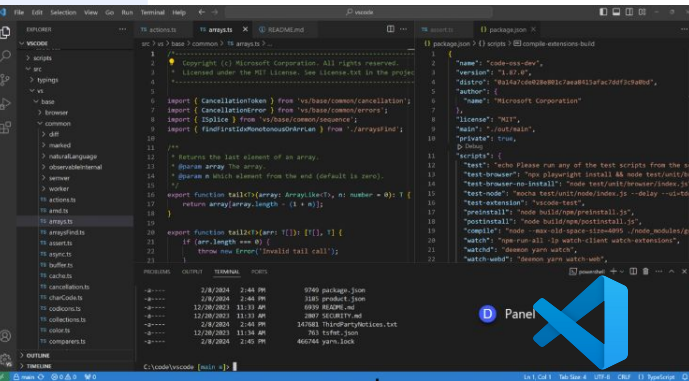
```
conda activate
```

```
conda info --envs
```

```
conda create -n myenv python=3.14
```



Integrated Development Environment (IDE)



Python introduction





SER-347/CAP-419

Pesquisar documentos

Introdução à Programação com Dados Geoespaciais

VISÃO GERAL DO CURSO:

Visão Geral do Curso

AULAS:

1. Instalando e Configurando o Ambiente de Programação

2. Introdução à Programação com a Linguagem Python

- 2.1. Introdução
- 2.2. Primeiro Programa em Python
- 2.3. Tipos de Dados
- 2.4. Tipos Numéricos
- 2.5. Expressões
- 2.6. Chamada de Funções
- 2.7. Variáveis
- 2.8. Comentários
- 2.9. Tipo Lógico
- 2.10. Operadores Relacionais
- 2.11. Expressões Lógicas
- 2.12. Estruturas Condicionais
- 2.13. Estruturas de Repetição

2. Introdução à Programação com a Linguagem Python

Anterior

Próximo

2. Introdução à Programação com a Linguagem Python

Um **programa** é formado por uma **seqüência de instruções** que diz ao computador como realizar uma determinada tarefa. As instruções podem dizer como o programa deve interagir com o usuário, como ele deve utilizar o hardware, e como operar os dados para realizar o que chamamos de **computações**. Quando um computador segue as instruções de um programa, dizemos que ele está **executando o programa**.

A atividade de **programar** consiste na habilidade de expressar as computações que dizem ao computador como resolver um determinado problema. Uma das ferramentas que permitem expressar essas computações são as **Linguagens de Programação**. O **Trecho de Código 2.1** exemplifica um programa escrito na linguagem Python, composto por uma seqüência de instruções encadeadas de forma a produzir o gráfico com a série temporal mostrado na Figura 2.1.

Trecho de Código 2.1 - Recuperando uma série temporal.

```
1 import matplotlib.pyplot as plt
2 from wtss import wtss
3
4 w = wtss('http://www.esensing.dpi.inpe.br/')
5
6 ts = w.time_series(coverage='M0013Q1', attributes=('red', 'nir'),
7                 latitude=-12.0, longitude=-54.0,
8                 start_date='2001-01-01', end_date='2001-12-31')
9
10 fig, ax = plt.subplots()
11
12 plt.title('Time Series M0013Q1', fontsize=24)
13
14 plt.xlabel('Date', fontsize=16)
15 plt.ylabel('Surface Reflectance', fontsize=16)
16
17 ax.plot(ts.timeline, ts['red'], color='red', ls='-', marker='o', label='red')
18 ax.plot(ts.timeline, ts['nir'], color='purple', ls='-', marker='o', label='nir')
19
20 plt.legend()
21
22 plt.grid(b=True, color='gray', linestyle='--', linewidth=0.5);
```

Python introduction

Q Search CTRL K

Introduction to GIS Programming

Course Information

Software Setup

Python Programming Fundamentals

Geospatial Programming with Python

Labs

Introduction to GIS Programming

Qiusheng Wu

Fall 2026 · Department of Geography and Sustainability · University of Tennessee

[Open in Canvas](#) [Search Syllabus](#) [Download Syllabus](#)

This is the course website for GEOG 312. It holds the lecture notebooks, lab assignments, syllabus, and schedule for the Fall 2026 offering. Everything here is public, so you are welcome to work through it even if you are not enrolled.

Course at a Glance

Meetings	Tuesdays and Thursdays, 12:55 to 2:10 p.m.
Location	Burchfiel Geography Building (BGB) 206
Term	August 17 to December 1, 2026
Credit	3 hours, letter graded
Instructor	Dr. Qiusheng Wu , BGB 309
Office hours	Wednesdays and Thursdays, 8:00 to 9:00 a.m., and by appointment
Email	qw18@utk.edu

Start with the [syllabus](#), also available as a [PDF](#), and the [weekly schedule](#).

What You Will Learn

GEOG 312 is a programming course taught through geography. No prior programming experience is assumed. The first half of the semester covers Python itself. The second half applies it to real geospatial data using the open source tools that working analysts and

CONTENTS

Course at a Glance

What You Will Learn

Course Materials

Textbook

Running the Code

Getting Help

License



Intro to GIS Programming

Python String Operations

1:08:19

Intro to GIS Programming

Python Functions and Classes

34:40

Intro to GIS Programming

Loops and Control Statements

55:14

Intro to GIS Programming

Files and Exception Handling

58:45

Intro to GIS Programming

Introduction to NumPy

45:28

Intro to GIS Programming

Introduction to NumPy and Pandas

1:02:07

Intro to GIS Programming | Week 3: Python String Operations

Open Geospatial Solutions • 1,2 mil • há 3 sem.

Intro to GIS Programming | Week 4: Python Functions and Classes

Open Geospatial Solutions • 1,5 mil • há 2 sem.

Intro to GIS Programming | Week 4: Loops and Conditional Statements

Open Geospatial Solutions • 689 • há 2 sem.

Intro to GIS Programming | Week 5: Working with Files

Open Geospatial Solutions • 1 mil • há 10 dias

Intro to GIS Programming | Week 6: Introduction to NumPy

Open Geospatial Solutions • 1 mil • há 3 dias

Intro to GIS Programming | Week 6: Introduction to NumPy and Pandas

Open Geospatial Solutions • 694 • há 1 dia

Open Geospatial Solutions

66,9 mil inscritos



Data Formats

Vector data

Format	Best suited for	Strengths	Limitations
Shapefile	Legacy data exchange	Very broad software support	Multiple files · size and attribute limitations
GeoPackage	General-purpose GIS	Single file · multiple layers · spatial index · open standard	More complex internal structure
GeoJSON	Web & APIs	Human-readable · simple · web-native	Large files · WGS 84-oriented
GeoParquet	Geo Data Science	Columnar · compressed · fast analytical I/O	Newer ecosystem

Other formats: FlatGeobuf · KML



JSON

JavaScript Object Notation (JSON)

JSON (JavaScript Object Notation) is a lightweight data-interchange format. It is easy for humans to read and write. It is easy for machines to parse and generate. It is based on a subset of the JavaScript Programming Language Standard ECMA-262 3rd Edition – December 1999. JSON is a text format that is completely language independent but uses conventions that are familiar to programmers of the C-family of languages, including C, C++, C#, Java, JavaScript, Perl, Python, and many others. These properties make JSON an ideal data-interchange language.

JSON

What is?

JSON

What is?

a textual format to represent and exchange structured data

```
{  
  "name": "Marujo",  
  "building": "SERE II",  
  "institution": "INPE"  
}
```

JSON

Which types does a JSON accepts?

JSON

Which types does a JSON accepts?

String, Number, Boolean, Null, Object, Array

```
{  
  "name": "Marujo",  
  "institution": "INPE",  
  "retired": false,  
  "pendencies": null,  
  "address": {  
    "building": "SERE II",  
    "room": 70  
  },  
  "disciplines": ["SER-347", "CAP-394", "CAP-423", "CAP-419", "SER-413"]  
}
```

JSON

```
with open('data.json', 'w', encoding='utf-8') as file:  
    json.dump(dictionary, file, indent=4, ensure_ascii=False)
```

Last executed at 2026-09-25 22:39:00 in 12ms

```
with open('data.json', 'r', encoding='utf-8') as file:  
    my_dict = json.load(file)
```

Last executed at 2026-09-25 22:39:00 in 13ms

JSON

```
with open('data.json', 'w', encoding='utf-8') as file:  
    json.dump(dictionary, file, indent=4, ensure_ascii=False)
```

Last executed at 2026-09-25 22:39:00 in 12ms

```
with open('data.json', 'r', encoding='utf-8') as file:  
    my_dict = json.load(file)
```

Last executed at 2026-09-25 22:39:00 in 13ms

load, loads, dump, dumps

JSON

```
json_str = """
{
    "name": "Marujo",
    "institution": "INPE",
    "retired": false,
    "pendencies": null,
    "address": {
        "building": "SERE II",
        "room": 70
    },
    "disciplines": ["SER-347", "CAP-394", "CAP-423", "CAP-419", "SER-413"]
}
"""

print(json_str)

data = json.loads(json_str)

print(data)

Last executed at 2026-09-25 22:39:00 in 20ms
```

load, loads, dump, dumps

The slide features a light blue background with decorative hexagonal shapes in the corners. The top-left corner has a cluster of overlapping hexagons in shades of light blue and cyan. The top-right corner has a similar cluster with a mix of light blue, cyan, and a slightly darker blue. The bottom-left and bottom-right corners have single, partially visible hexagons in light blue and cyan respectively.

JSON

What is JSON relationship with JavaScript?

JSON

Why has JSON become so popular?



GEOJSON

GEOJSON

```
{  
  "type": "Point",  
  "coordinates": [-45.88, -23.18]  
}
```

```
{  
  "type": "LineString",  
  "coordinates": [  
    [-45.88, -23.18],  
    [-45.90, -23.20],  
    [-45.92, -23.22]  
  ]  
}
```

```
{  
  "type": "Polygon",  
  "coordinates": [  
    [  
      [-45.88, -23.18],  
      [-45.90, -23.18],  
      [-45.90, -23.20],  
      [-45.88, -23.20],  
      [-45.88, -23.18]  
    ]  
  ]  
}
```

GEOJSON

```
{
  "type": "Feature",
  "geometry": {
    "type": "Point",
    "coordinates": [-45.88, -23.18]
  },
  "properties": {
    "nome": "INPE",
    "tipo": "Instituição de
pesquisa"
  }
}
```

```
{
  "type": "FeatureCollection",
  "features": [
    {
      "type": "Feature",
      "geometry": {
        "type": "Point",
        "coordinates": [-45.88, -23.18]
      },
      "properties": {
        "nome": "INPE"
      }
    },
    {
      "type": "Feature",
      "geometry": {
        "type": "Point",
        "coordinates": [-46.63, -23.55]
      },
      "properties": {
        "nome": "São Paulo"
      }
    }
  ]
}
```



Packages

Packages

Numpy

GeoPandas

Pandas

OGR

Shapely

GDAL

Fiona

Rasterio

Packages

Numpy

1.1 Dimensions and shape

1.2 dtype

1.3 Indexing and slicing

1.4 Boolean masks

1.5 Vectorized operations and broadcasting

1.6 Statistics

Packages

Pandas

2.1 Essential operations: a first look at the data

2.2 Selection

2.3 Filtering

2.4 Creating attributes (new columns)

2.5 Grouping (split → apply → combine)

2.6 Joining tables

2.7 Missing values

2.8 Read / write

Packages

Shapely

Fiona

GeoPandas

Reading and writing vector files

Area

Buffer

Centroids

Spatial join

Packages

OGR

Packages

OGR

ogrinfo

ogr2ogr

GDAL

Packages

OGR

ogrinfo
ogr2ogr

GDAL

gdalinfo
gdal_translate
gdal_warp
gdal_merge
gdal_calc

Packages

Rasterio

7.1 Opening a dataset and reading its metadata

7.2 read

7.3 masked

7.4 Window



NumPy



Numpy

nature

[View](#)

[Explore content](#) ▾

[About the journal](#) ▾

[Publish with us](#) ▾

[nature](#) > [review articles](#) > [article](#)

Review Article | [Open access](#) | Published: 16 September 2020

Array programming with NumPy

[Charles R. Harris](#), [K. Jarrod Millman](#) ✉, [Stéfan J. van der Walt](#) ✉, [Ralf Gommers](#) ✉, [Pauli Virtanen](#), [David Cournapeau](#), [Eric Wieser](#), [Julian Taylor](#), [Sebastian Berg](#), [Nathaniel J. Smith](#), [Robert Kern](#), [Matti Picus](#), [Stephan Hoyer](#), [Marten H. van Kerkwijk](#), [Matthew Brett](#), [Allan Haldane](#), [Jaime Fernández del Río](#), [Mark Wiebe](#), [Pearu Peterson](#), [Pierre Gérard-Marchant](#), [Kevin Sheppard](#), [Tyler Reddy](#), [Warren Weckesser](#), [Hameer Abbasi](#), ... [Travis E. Oliphant](#) [+ Show authors](#)

[Nature](#) **585**, 357–362 (2020) | [Cite this article](#)

[Save article](#)

710k Accesses | **29k** Citations | **1645** Altmetric | [Metrics](#)

Abstract

Numpy

1.1 Dimensions and shape

```
a = np.array([1, 2, 3, 4])           # 1D: e.g. a time series of one pixel
m = np.array([[1, 2, 3],
               [4, 5, 6]])           # 2D: a single-band image (rows, cols)
rng = np.random.default_rng(42)
cube = rng.integers(0, 255, size=(3, 4, 5)) # 3D: a multiband image (bands, rows, cols)

for name, arr in [("a", a), ("m", m), ("cube", cube)]:
    print(f"{name:>4}: ndim={arr.ndim} shape={arr.shape} size={arr.size} dtype={arr.dtype}")
```

Last executed at 2026-09-25 22:39:00 in 87ms

```
  a: ndim=1  shape=(4,)  size=4  dtype=int64
  m: ndim=2  shape=(2, 3)  size=6  dtype=int64
cube: ndim=3  shape=(3, 4, 5)  size=60  dtype=int64
```

Numpy

1.2 dtype

```
img = np.array([[250, 10],  
                [100, 200]], dtype=np.uint8)  
print(img.dtype, "->", img.itemsize, "byte per element")  
  
print("img + 10 (uint8 overflow wraps around!):\n", img + 10)  
print("img.astype(np.int16) + 10:\n", img.astype(np.int16) + 10)  
Last executed at 2026-09-25 22:39:01 in 15ms
```

Numpy

1.2 dtype

```
img = np.array([[250, 10],  
               [100, 200]], dtype=np.uint8)  
print(img.dtype, "->", img.itemsize, "byte per element")  
  
print("img + 10 (uint8 overflow wraps around!):\n", img + 10)  
print("img.astype(np.int16) + 10:\n", img.astype(np.int16) + 10)
```

Last executed at 2026-09-25 22:39:01 in 15ms

```
uint8 -> 1 byte per element  
img + 10 (uint8 overflow wraps around!):  
[[ 4 20]  
 [110 210]]  
img.astype(np.int16) + 10:  
[[260 20]  
 [110 210]]
```

Numpy

1.3 Indexing and slicing

```
z = np.arange(48).reshape(6, 8)  # a small 6 x 8 "image"
print(z)
```

Last executed at 2026-09-25 22:39:01 in 14ms

```
[[ 0  1  2  3  4  5  6  7]
 [ 8  9 10 11 12 13 14 15]
 [16 17 18 19 20 21 22 23]
 [24 25 26 27 28 29 30 31]
 [32 33 34 35 36 37 38 39]
 [40 41 42 43 44 45 46 47]]
```

```
print("single pixel z[2, 3]      :", z[2, 3])
print("last pixel z[-1, -1]     :", z[-1, -1])
print("row 1 z[1]                :", z[1])
print("column 3 z[:, 3]         :", z[:, 3])
print("window z[1:4, 2:5]       :\n", z[1:4, 2:5])
print("every 2nd pixel z[::2, ::2] (a cheap downsampling):\n", z[::2, ::2])
```

Last executed at 2026-09-25 22:39:01 in 12ms

```
single pixel z[2, 3]      : 19
last pixel z[-1, -1]     : 47
row 1 z[1]                : [ 8  9 10 11 12 13 14 15]
column 3 z[:, 3]         : [ 3 11 19 27 35 43]
window z[1:4, 2:5]       :
[[10 11 12]
 [18 19 20]
 [26 27 28]]
every 2nd pixel z[::2, ::2] (a cheap downsampling):
[[ 0  2  4  6]
 [16 18 20 22]
 [32 34 36 38]]
```

Numpy

Views vs. copies

```
z = np.arange(48).reshape(6, 8)
view = z[0:2, 0:2]
view[:] = -1
print(z[:3, :4])  # the original changed!

z = np.arange(48).reshape(6, 8)
copy = z[0:2, 0:2].copy()
copy[:] = -1
print(z[:3, :4])  # the original is untouched
```

Last executed at 2026-09-25 22:39:01 in 13ms

```
[[ -1  -1   2   3]
 [ -1  -1  10  11]
 [16  17  18  19]]
[[ 0  1  2  3]
 [ 8  9 10 11]
 [16 17 18 19]]
```

Numpy

1.4 Boolean masks

```
rng = np.random.default_rng(0)
temp = rng.normal(25, 5, size=(5, 6)).round(1) # e.g. a land surface temperature grid (°C)
print(temp)

hot = temp > 30
print(hot)
print("values above 30 °C:", temp[hot])
print("number of hot pixels:", hot.sum(), f"({hot.mean():.0%} of the grid)")

Last executed at 2026-09-25 22:39:01 in 14ms

[[25.6 24.3 28.2 25.5 22.3 26.8]
 [31.5 29.7 21.5 18.7 21.9 25.2]
 [13.4 23.9 18.8 21.3 22.3 23.4]
 [27.1 30.2 24.4 31.8 21.7 26.8]
 [29.5 25.5 21.3 20.4 22.7 26.1]]
[[False False False False False False]
 [ True False False False False False]
 [False False False False False False]
 [False  True False  True False False]
 [False False False False False False]]
values above 30 °C: [31.5 30.2 31.8]
number of hot pixels: 3 (10% of the grid)

# Combine conditions with & (and), | (or), ~ (not) -- parentheses are mandatory!
mild = (temp >= 20) & (temp <= 30)
print("mild pixels:", mild.sum())

# np.where: vectorized if/else -> a simple classification
classes = np.where(temp > 30, 2, np.where(temp < 20, 0, 1))
print(classes)

Last executed at 2026-09-25 22:39:01 in 12ms

mild pixels: 24
[[1 1 1 1 1]
 [2 1 1 0 1]
 [0 1 0 1 1]
 [1 2 1 2 1]
 [1 1 1 1 1]]
```

Numpy

1.5 Vectorized operations

$$NDVI = \frac{NIR - Red}{NIR + Red}$$

```
rng = np.random.default_rng(1)
red = rng.uniform(0.02, 0.30, size=(1000, 1000)).astype(np.float32)
nir = rng.uniform(0.10, 0.60, size=(1000, 1000)).astype(np.float32)

def ndvi_loop(nir, red):
    out = np.empty_like(nir)
    for i in range(nir.shape[0]):
        for j in range(nir.shape[1]):
            out[i, j] = (nir[i, j] - red[i, j]) / (nir[i, j] + red[i, j])
    return out

%time ndvi_l = ndvi_loop(nir, red)
%time ndvi = (nir - red) / (nir + red)
print("same result?", np.allclose(ndvi, ndvi_l))

Last executed at 2026-09-25 22:39:01 in 777ms

CPU times: user 725 ms, sys: 0 ns, total: 725 ms
Wall time: 723 ms
CPU times: user 9.19 ms, sys: 0 ns, total: 9.19 ms
Wall time: 8.59 ms
```

Numpy

1.6 Statistics

The `axis` argument chooses **along which dimension** we aggregate:

- `axis=None` → the whole array
- `axis=0` on (bands, rows, cols) → one value per pixel, aggregating over bands (or over **time** in a time series)
- `axis=(1, 2)` → one value per band

```
print(f"min={ndvi.min():.3f} max={ndvi.max():.3f} mean={ndvi.mean():.3f} std={ndvi.std():.3f}")
print("median      :", np.median(ndvi))
print("2nd/98th pctl  :", np.percentile(ndvi, [2, 98])) # common for contrast stretching

print("per band mean  :", cube.mean(axis=(1, 2)).round(1))
print("per pixel mean :", cube.mean(axis=0).shape)

counts, edges = np.histogram(ndvi, bins=10, range=(-1, 1))
print("histogram counts:", counts)
```

Last executed at 2026-09-25 22:39:02 in 61ms

```
min=-0.499 max=0.935 mean=0.356 std=0.309
median      : 0.37272704
2nd/98th pctl  : [-0.32379147  0.87767315]
per band mean  : [4940.  4972.8 4992.9 4967.2]
per pixel mean : (100, 100)
histogram counts: [  0      0  6786  46890  89245 148860 243571 223714 171020 69914]
```

```
fig, axes = plt.subplots(1, 2, figsize=(11, 4))
im = axes[0].imshow(ndvi, cmap="RdYlGn", vmin=-1, vmax=1)
axes[0].set_title("NDVI (random data)")
fig.colorbar(im, ax=axes[0])
axes[1].hist(ndvi.ravel(), bins=50, color="seagreen")
axes[1].set_title("NDVI histogram")
plt.tight_layout()
```

Last executed at 2026-09-25 22:39:02 in 842ms

QUIZ

Why don't use lists instead of numpy?



Pandas



<https://prog-geo.github.io/vetorial/pandas.html>

Pandas

```
capitals = {
    "city": ["São Paulo", "Rio de Janeiro", "Brasília", "Fortaleza", "Salvador", "Belo Horizonte",
            "Manaus", "Curitiba", "Recife", "Goiânia", "Belém", "Porto Alegre", "São Luís",
            "Maceió", "Campo Grande", "Teresina", "João Pessoa", "Natal", "Cuiabá", "Aracaju",
            "Florianópolis", "Porto Velho", "Macapá", "Rio Branco", "Vitória", "Boa Vista", "Palmas"],
    "state": ["SP", "RJ", "DF", "CE", "BA", "MG", "AM", "PR", "PE", "GO", "PA", "RS", "MA", "AL", "MS",
            "PI", "PB", "RN", "MT", "SE", "SC", "RO", "AP", "AC", "ES", "RR", "TO"],
    "region": ["Sudeste", "Sudeste", "Centro-Oeste", "Nordeste", "Nordeste", "Sudeste", "Norte", "Sul",
            "Nordeste", "Centro-Oeste", "Norte", "Sul", "Nordeste", "Nordeste", "Centro-Oeste",
            "Nordeste", "Nordeste", "Nordeste", "Centro-Oeste", "Nordeste", "Sul", "Norte", "Norte",
            "Norte", "Sudeste", "Norte", "Norte"],
    "lat": [-23.55, -22.91, -15.79, -3.73, -12.97, -19.92, -3.12, -25.43, -8.05, -16.68, -1.46,
            -30.03, -2.53, -9.67, -20.47, -5.09, -7.12, -5.79, -15.60, -10.91, -27.59, -8.76,
            0.03, -9.97, -20.32, 2.82, -10.18],
    "lon": [-46.63, -43.17, -47.88, -38.52, -38.50, -43.94, -60.02, -49.27, -34.88, -49.25, -48.49,
            -51.23, -44.30, -35.74, -54.62, -42.80, -34.86, -35.21, -56.10, -37.07, -48.55, -63.90,
            -51.07, -67.81, -40.34, -60.67, -48.33],
    "population": [11451999, 6211223, 2817381, 2428708, 2418005, 2315560, 2063689, 1773718, 1488920,
            1437366, 1303403, 1332845, 1037775, 957916, 898100, 866300, 833932, 751300,
            650877, 602757, 537211, 460434, 442933, 364756, 322869, 413486, 302692],
    "area_km2": [1521, 1200, 5760, 312, 693, 331, 11401, 435, 218, 729, 1059, 495, 583, 509, 8083,
            1391, 211, 167, 4327, 182, 675, 34091, 6563, 8835, 97, 5687, 2219],
    "elevation_m": [760, 5, 1172, 21, 8, 852, 92, 934, 4, 749, 10, 10, 4, 16, 592, 72, 37, 30, 165,
            4, 3, 85, None, 153, 3, None, None], # some values were "not collected"
}

pd.DataFrame(capitals).to_csv(DATA / "brazil_capitals.csv", index=False)

df = pd.read_csv(DATA / "brazil_capitals.csv")
df
```

Last executed at 2026-09-25 22:39:02 in 38ms

Pandas

	city	state	region	lat	lon	population	area_km2	elevation_m
0	São Paulo	SP	Sudeste	-23.55	-46.63	11451999	1521	760.0
1	Rio de Janeiro	RJ	Sudeste	-22.91	-43.17	6211223	1200	5.0
2	Brasília	DF	Centro-Oeste	-15.79	-47.88	2817381	5760	1172.0
3	Fortaleza	CE	Nordeste	-3.73	-38.52	2428708	312	21.0
4	Salvador	BA	Nordeste	-12.97	-38.50	2418005	693	8.0
5	Belo Horizonte	MG	Sudeste	-19.92	-43.94	2315560	331	852.0
6	Manaus	AM	Norte	-3.12	-60.02	2063689	11401	92.0
7	Curitiba	PR	Sul	-25.43	-49.27	1773718	435	934.0
8	Recife	PE	Nordeste	-8.05	-34.88	1488920	218	4.0
9	Goiânia	GO	Centro-Oeste	-16.68	-49.25	1437366	729	749.0
10	Belém	PA	Norte	-1.46	-48.49	1303403	1059	10.0
11	Porto Alegre	RS	Sul	-30.03	-51.23	1332845	495	10.0
12	São Luís	MA	Nordeste	-2.53	-44.30	1037775	583	4.0
13	Maceió	AL	Nordeste	-9.67	-35.74	957916	509	16.0
14	Campo Grande	MS	Centro-Oeste	-20.47	-54.62	898100	8083	592.0
15	Teresina	PI	Nordeste	-5.09	-42.80	866300	1391	72.0
16	João Pessoa	PB	Nordeste	-7.12	-34.86	833932	211	37.0
17	Natal	RN	Nordeste	-5.79	-35.21	751300	167	30.0
18	Cuiabá	MT	Centro-Oeste	-15.60	-56.10	650877	4327	165.0
19	Aracaju	SE	Nordeste	-10.91	-37.07	602757	182	4.0
20	Florianópolis	SC	Sul	-27.59	-48.55	537211	675	3.0
21	Porto Velho	RO	Norte	-8.76	-63.90	460434	34091	85.0
22	Macapá	AP	Norte	0.03	-51.07	442933	6563	NaN
23	Rio Branco	AC	Norte	-9.97	-67.81	364756	8835	153.0
24	Vitória	ES	Sudeste	-20.32	-40.34	322869	97	3.0
25	Boa Vista	RR	Norte	2.82	-60.67	413486	5687	NaN
26	Palmas	TO	Norte	-10.18	-48.33	302692	2219	NaN

Pandas

2.1 Essential operations: a first look at the data

```
df.head() # first 5 rows (df.head(10) for 10); df.tail() for the last ones
```

Last executed at 2026-09-25 22:39:02 in 12ms

	city	state	region	lat	lon	population	area_km2	elevation_m
0	São Paulo	SP	Sudeste	-23.55	-46.63	11451999	1521	760.0
1	Rio de Janeiro	RJ	Sudeste	-22.91	-43.17	6211223	1200	5.0
2	Brasília	DF	Centro-Oeste	-15.79	-47.88	2817381	5760	1172.0
3	Fortaleza	CE	Nordeste	-3.73	-38.52	2428708	312	21.0
4	Salvador	BA	Nordeste	-12.97	-38.50	2418005	693	8.0

Pandas

2.1 Essential operations: a first look at the data

```
df.info()           # column names, dtypes, non-null counts, memory usage
Last executed at 2026-09-25 22:39:02 in 19ms
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 27 entries, 0 to 26
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   city             27 non-null    object
1   state            27 non-null    object
2   region           27 non-null    object
3   lat              27 non-null    float64
4   lon              27 non-null    float64
5   population       27 non-null    int64
6   area_km2         27 non-null    int64
7   elevation_m      24 non-null    float64
dtypes: float64(3), int64(2), object(3)
memory usage: 1.8+ KB
```

Pandas

2.1 Essential operations: a first look at the data

<code>df.describe()</code> <i># summary statistics of numeric columns</i>					
Last executed at 2026-09-25 22:39:02 in 28ms					
	lat	lon	population	area_km2	elevation_m
count	27.00000	27.000000	2.700000e+01	27.000000	24.000000
mean	-12.39963	-47.153704	1.721709e+06	3621.259259	240.875000
std	9.01520	9.121861	2.292924e+06	6859.651626	369.678981
min	-30.03000	-67.810000	3.026920e+05	97.000000	3.000000
25%	-20.12000	-51.150000	5.699840e+05	383.000000	7.250000
50%	-10.18000	-47.880000	9.579160e+05	729.000000	33.500000
75%	-5.44000	-39.430000	1.918704e+06	5007.000000	271.750000
max	2.82000	-34.860000	1.145200e+07	34091.000000	1172.000000

Pandas

2.1 Essential operations: a first look at the data

```
print("shape :", df.shape)           # (rows, columns)
print("columns:", df.columns.tolist())
print(df.dtypes)
df.describe(exclude="number")         # summary of the non-numeric (text) columns
```

Last executed at 2026-09-25 22:39:03 in 17ms

```
shape : (27, 8)
columns: ['city', 'state', 'region', 'lat', 'lon', 'population', 'area_km2', 'elevation_m']
city      object
state     object
region    object
lat       float64
lon       float64
population int64
area_km2  int64
elevation_m float64
dtype: object
```

	city	state	region
count	27	27	27
unique	27	27	5
top	São Paulo	SP	Nordeste
freq	1	1	9

Pandas

2.2 Selection

```
df["city"].head(3)
```

Last executed at 2026-09-25 22:39:03 in 11ms

```
0      São Paulo
1    Rio de Janeiro
2      Brasília
Name: city, dtype: object
```

```
df[["city", "population"]].head(3)
```

Last executed at 2026-09-25 22:39:03 in 16ms

	city	population
0	São Paulo	11451999
1	Rio de Janeiro	6211223
2	Brasília	2817381

```
print(df.iloc[0])          # first row, by position
df.iloc[:3, :4]           # first 3 rows, first 4 columns
```

Last executed at 2026-09-25 22:39:03 in 24ms

city	São Paulo
state	SP
region	Sudeste
lat	-23.55
lon	-46.63
population	11451999
area_km2	1521
elevation_m	760.0

Name: 0, dtype: object

	city	state	region	lat
0	São Paulo	SP	Sudeste	-23.55
1	Rio de Janeiro	RJ	Sudeste	-22.91
2	Brasília	DF	Centro-Oeste	-15.79

```
dfc = df.set_index("city")          # use the city name as the row label
print(dfc.loc["Manaus", "population"])
dfc.loc[["Manaus", "Belém"], ["state", "population", "area_km2"]]
```

Last executed at 2026-09-25 22:39:03 in 26ms

2063689			
	state	population	area_km2
	city		
Manaus	AM	2063689	11401
Belém	PA	1303403	1059

Pandas

2.3 Filtering

```
df[df["population"] > 2_000_000]
```

Last executed at 2026-09-25 22:39:03 in 20ms

	city	state	region	lat	lon	population	area_km2	elevation_m
0	São Paulo	SP	Sudeste	-23.55	-46.63	11451999	1521	760.0
1	Rio de Janeiro	RJ	Sudeste	-22.91	-43.17	6211223	1200	5.0
2	Brasília	DF	Centro-Oeste	-15.79	-47.88	2817381	5760	1172.0
3	Fortaleza	CE	Nordeste	-3.73	-38.52	2428708	312	21.0
4	Salvador	BA	Nordeste	-12.97	-38.50	2418005	693	8.0
5	Belo Horizonte	MG	Sudeste	-19.92	-43.94	2315560	331	852.0
6	Manaus	AM	Norte	-3.12	-60.02	2063689	11401	92.0

Pandas

2.3 Filtering

```
# Several conditions: & | ~ and parentheses  
df[(df["region"] == "Nordeste") & (df["population"] > 1_000_000)]
```

Last executed at 2026-09-25 22:39:03 in 17ms

	city	state	region	lat	lon	population	area_km2	elevation_m
3	Fortaleza	CE	Nordeste	-3.73	-38.52	2428708	312	21.0
4	Salvador	BA	Nordeste	-12.97	-38.50	2418005	693	8.0
8	Recife	PE	Nordeste	-8.05	-34.88	1488920	218	4.0
12	São Luís	MA	Nordeste	-2.53	-44.30	1037775	583	4.0

```
print(df[df["state"].isin(["SP", "RJ", "MG", "ES"])]["city"].tolist())  
print(df[df["city"].str.startswith("São")]["city"].tolist())
```

```
# .query() is often more readable  
df.query("region == 'Sul' and elevation_m > 500")
```

Last executed at 2026-09-25 22:39:03 in 23ms

```
['São Paulo', 'Rio de Janeiro', 'Belo Horizonte', 'Vitória']  
['São Paulo', 'São Luís']
```

	city	state	region	lat	lon	population	area_km2	elevation_m
7	Curitiba	PR	Sul	-25.43	-49.27	1773718	435	934.0

Pandas

2.4 Creating attributes (new columns)

```
df["density"] = df["population"] / df["area_km2"]           # vectorized, like NumPy
df["pop_millions"] = (df["population"] / 1e6).round(2)
df["hemisphere"] = np.where(df["lat"] >= 0, "North", "South")
df["size_class"] = pd.cut(df["population"],
                           bins=[0, 500_000, 1_000_000, 2_000_000, np.inf],
                           labels=["< 500k", "500k-1M", "1M-2M", "> 2M"])
df["label"] = df["city"] + " (" + df["state"] + ")"

df.sort_values("density", ascending=False).head()[["label", "population", "area_km2", "density", "size_class"]]
```

Last executed at 2026-09-25 22:39:03 in 30ms

	label	population	area_km2	density	size_class
3	Fortaleza (CE)	2428708	312	7784.320513	> 2M
0	São Paulo (SP)	11451999	1521	7529.256410	> 2M
5	Belo Horizonte (MG)	2315560	331	6995.649547	> 2M
8	Recife (PE)	1488920	218	6829.908257	1M-2M
1	Rio de Janeiro (RJ)	6211223	1200	5176.019167	> 2M

Pandas

2.5 Grouping (split → apply → combine)

```
df.groupby("region")["population"].sum().sort_values(ascending=False)
```

Last executed at 2026-09-25 22:39:03 in 13ms

```
region
Sudeste      20301651
Nordeste     11385613
Centro-Oeste  5803724
Norte        5351393
Sul          3643774
Name: population, dtype: int64
```

```
summary = df.groupby("region").agg(
    n_capitals=("city", "count"),
    total_pop=("population", "sum"),
    mean_density=("density", "mean"),
    max_elevation=("elevation_m", "max"),
)
summary
```

Last executed at 2026-09-25 22:39:03 in 27ms

	n_capitals	total_pop	mean_density	max_elevation
region				
Centro-Oeste	4	5803724	680.589022	1172.0
Nordeste	9	11385613	3794.573117	72.0
Norte	7	5351393	249.027609	153.0
Sudeste	4	20301651	5757.367879	852.0
Sul	3	3643774	2521.998984	934.0

```
# transform: returns a value per ROW (same size as df) -> e.g. the share of each capital in its region
df["share_of_region"] = df["population"] / df.groupby("region")["population"].transform("sum")
df[["city", "region", "share_of_region"]].sort_values("share_of_region", ascending=False).head()
```

Last executed at 2026-09-25 22:39:03 in 17ms

	city	region	share_of_region
0	São Paulo	Sudeste	0.564092
7	Curitiba	Sul	0.486780
2	Brasília	Centro-Oeste	0.485444
6	Manaus	Norte	0.385636
11	Porto Alegre	Sul	0.365787

Pandas

2.6 Joining tables

```
regions = pd.DataFrame({
    "region": ["Norte", "Nordeste", "Centro-Oeste", "Sudeste", "Sul"],
    "region_area_km2": [3_850_000, 1_552_000, 1_606_000, 924_600, 576_800],
    "n_states": [7, 9, 4, 4, 3],
})
```

```
merged = df.merge(regions, on="region", how="left", validate="many_to_one")
merged[["city", "region", "region_area_km2", "n_states"]].head()
```

Last executed at 2026-09-25 22:39:03 in 16ms

	city	region	region_area_km2	n_states
0	São Paulo	Sudeste	924600	4
1	Rio de Janeiro	Sudeste	924600	4
2	Brasília	Centro-Oeste	1606000	4
3	Fortaleza	Nordeste	1552000	9
4	Salvador	Nordeste	1552000	9

```
# how= "inner" | "left" | "right" | "outer" -> what happens with keys that do not match?
extra = pd.DataFrame({"region": ["Sul", "Antarctica"], "note": ["cold", "very cold"]})
print(df.merge(extra, on="region", how="inner").shape)
print(df.merge(extra, on="region", how="left").shape)
print(df.merge(extra, on="region", how="outer").shape)
```

```
# pd.concat stacks tables (rows or columns)
pd.concat([df.head(2), df.tail(2)]).loc[["city", "state"]]
```

Last executed at 2026-09-25 22:39:03 in 28ms

```
(3, 15)
(27, 15)
(28, 15)
```

	city	state
0	São Paulo	SP
1	Rio de Janeiro	RJ
25	Boa Vista	RR
26	Palmas	TO

Pandas

2.7 Missing values

```
print(df.isna().sum())
df[df["elevation_m"].isna()][["city", "state", "elevation_m"]]
```

Last executed at 2026-09-25 22:39:03 in 18ms

city	0
state	0
region	0
lat	0
lon	0
population	0
area_km2	0
elevation_m	3
density	0
pop_millions	0
hemisphere	0
size_class	0
label	0
share_of_region	0

dtype: int64

	city	state	elevation_m
22	Macapá	AP	NaN
25	Boa Vista	RR	NaN
26	Palmas	TO	NaN

```
print("rows before dropna:", len(df), " after:", len(df.dropna(subset=["elevation_m"])))
```

```
# Fill with a constant, or with a statistic of the group (here: median elevation of the region)
df["elevation_filled"] = df["elevation_m"].fillna(
    df.groupby("region")["elevation_m"].transform("median")
)
df[df["elevation_m"].isna()][["city", "region", "elevation_m", "elevation_filled"]]
```

Last executed at 2026-09-25 22:39:03 in 24ms

rows before dropna: 27 after: 24

	city	region	elevation_m	elevation_filled
22	Macapá	Norte	NaN	88.5
25	Boa Vista	Norte	NaN	88.5
26	Palmas	Norte	NaN	88.5

Pandas

2.8 Read / write

Format	Read	Write	Notes
CSV	<code>pd.read_csv</code>	<code>df.to_csv</code>	universal, but no types, large and slow
Excel	<code>pd.read_excel</code>	<code>df.to_excel</code>	needs <code>openpyxl</code>
Parquet	<code>pd.read_parquet</code>	<code>df.to_parquet</code>	columnar, compressed, typed — recommended for large data
SQL	<code>pd.read_sql</code>	<code>df.to_sql</code>	databases (PostgreSQL/PostGIS, SQLite, ...)

```
df.to_csv(OUT / "capitals_enriched.csv", index=False)
df.to_parquet(OUT / "capitals_enriched.parquet")

for f in ["capitals_enriched.csv", "capitals_enriched.parquet"]:
    print(f"{f:28s} {os.path.getsize(OUT / f):>7,d} bytes")

back = pd.read_parquet(OUT / "capitals_enriched.parquet")
print(back.dtypes[["size_class", "density"]])  # Parquet keeps the dtypes (CSV would not)
```

Last executed at 2026-09-25 22:39:03 in 78ms

capitals_enriched.csv	3,566 bytes
capitals_enriched.parquet	12,029 bytes
size_class	category
density	float64
dtype:	object



GeoPandas



Shapely

3.1 Shapely

```
import shapely
from shapely.geometry import Point, LineString, Polygon

sjc = Point(-45.88, -23.18)          # São José dos Campos -- ATTENTION: (x=Lon, y=Lat)
line = LineString([(0, 0), (1, 1), (2, 0)])
square = Polygon([(0, 0), (2, 0), (2, 2), (0, 2)])

print("shapely", shapely.__version__)
print(sjc.wkt)
print("line length :", line.length)
print("square area :", square.area)
print("contains   :", square.contains(Point(1, 1)))
print("intersects :", square.intersects(line))
print("buffer area :", round(Point(0, 0).buffer(1).area, 4), "(≈ pi: a polygon approximating a circle)")
square
```

Last executed at 2026-09-25 22:39:03 in 123ms

```
shapely 2.1.1
POINT (-45.88 -23.18)
line length : 2.8284271247461903
square area : 4.0
contains    : True
intersects  : True
buffer area : 3.1365 (≈ pi: a polygon approximating a circle)
```

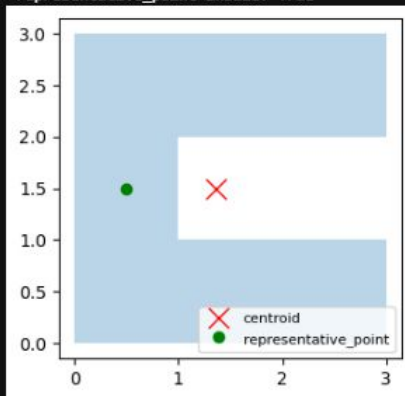


```
# centroid vs representative_point: the centroid of a concave polygon may fall OUTSIDE it
c_shape = Polygon([(0, 0), (3, 0), (3, 1), (1, 1), (1, 2), (3, 2), (3, 3), (0, 3)])
print("centroid inside?", c_shape.contains(c_shape.centroid))
print("representative_point inside?", c_shape.contains(c_shape.representative_point()))
```

```
fig, ax = plt.subplots(figsize=(3.5, 3.5))
ax.fill(*c_shape.exterior.xy, alpha=0.3)
ax.plot(*c_shape.centroid.xy, "rx", ms=12, label="centroid")
ax.plot(*c_shape.representative_point().xy, "go", label="representative_point")
ax.legend(loc="lower right", fontsize=8);
```

Last executed at 2026-09-25 22:39:03 in 137ms

```
centroid inside?      False
representative_point inside? True
```



GeoPandas

3.2 GeoPandas

A table with coordinates is **not yet** spatial data. We need:

1. a **geometry** column (Shapely objects);
2. a **CRS** telling what the coordinates mean.

Our capitals have `lon` / `lat` in degrees on WGS 84 → **EPSG:4326**.

```
import geopandas as gpd
import pyproj

print("geopandas", gpd.__version__, "| pyproj", pyproj.__version__, "| PROJ", pyproj.proj_version_str)

gdf = gpd.GeoDataFrame(
    df,
    geometry=gpd.points_from_xy(df["lon"], df["lat"]), # x = lon, y = lat !
    crs="EPSG:4326",
)
gdf[["city", "state", "population", "geometry"]].head()
```

Last executed at 2026-09-25 22:39:03 in 74ms

geopandas 1.1.1 | pyproj 3.7.2 | PROJ 9.5.1

	city	state	population	geometry
0	São Paulo	SP	11451999	POINT (-46.63 -23.55)
1	Rio de Janeiro	RJ	6211223	POINT (-43.17 -22.91)
2	Brasília	DF	2817381	POINT (-47.88 -15.79)
3	Fortaleza	CE	2428708	POINT (-38.52 -3.73)
4	Salvador	BA	2418005	POINT (-38.5 -12.97)

GeoPandas

3.2 GeoPandas

```
# Another common case: geometries stored as WKT text (e.g. from a database or a CSV export)
wkt_table = pd.DataFrame({
    "name": ["INPE - São José dos Campos", "Paraíba do Sul (segment)", "Study area"],
    "wkt": ["POINT (-45.86 -23.21)",
           "LINESTRING (-46.0 -23.3, -45.9 -23.2, -45.7 -23.1)",
           "POLYGON ((-46 -23.4, -45.6 -23.4, -45.6 -23.0, -46 -23.0, -46 -23.4))"],
})
wkt_gdf = gpd.GeoDataFrame(wkt_table, geometry=gpd.GeoSeries.from_wkt(wkt_table["wkt"]), crs="EPSG:4326")
wkt_gdf
```

Last executed at 2026-09-25 22:39:03 in 10ms

	name	wkt	geometry
0	INPE - São José dos Campos	POINT (-45.86 -23.21)	POINT (-45.86 -23.21)
1	Paraíba do Sul (segment)	LINESTRING (-46.0 -23.3, -45.9 -23.2, -45.7 -23.1)	LINESTRING (-46 -23.3, -45.9 -23.2, -45.7 -23.1)
2	Study area	POLYGON ((-46 -23.4, -45.6 -23.4, -45.6 -23.0, -46 -23.0, -46 -23.4))	POLYGON ((-46 -23.4, -45.6 -23.4, -45.6 -23.0, -46 -23.0, -46 -23.4))

GeoPandas

3.3 GeoDataFrame, GeoSeries and geometry

- **GeoDataFrame** : a DataFrame with (at least) one geometry column — the *active* geometry.
- **GeoSeries** : a Series of geometries (+ CRS). Spatial methods (`area`, `buffer`, `centroid`, ...) live here.
- `gdf.geometry` : always returns the active geometry column, whatever its name.

```
print(type(gdf))
print(type(gdf.geometry), "| active geometry column:", gdf.geometry.name)
print(type(gdf["population"]))
print("geometry types:", gdf.geom_type.unique().tolist())
print("total bounds  :", gdf.total_bounds)          # [minx, miny, maxx, maxy]
gdf.geometry.x.head(3), gdf.geometry.y.head(3)
```

Last executed at 2026-09-25 22:39:04 in 26ms

```
<class 'geopandas.geodataframe.GeoDataFrame'>
<class 'geopandas.geoseries.GeoSeries'> | active geometry column: geometry
<class 'pandas.core.series.Series'>
geometry types: ['Point']
total bounds  : [-67.81 -30.03 -34.86  2.82]

(0  -46.63
 1  -43.17
 2  -47.88
dtype: float64,
0  -23.55
 1  -22.91
 2  -15.79
dtype: float64)
```

GeoPandas

3.4 Reading and writing vector files

GeoPandas reads/writes (through GDAL/OGR) virtually any vector format.

Format	Extension	Comments
GeoPackage	<code>.gpkg</code>	SQLite based, single file, multiple layers — recommended default
Shapefile	<code>.shp</code> + <code>.shx</code> + <code>.dbf</code> + <code>.prj</code> ...	legacy: 10-char column names, 2 GB limit, several files
GeoJSON	<code>.geojson</code>	text, web-friendly, always WGS 84
GeoParquet	<code>.parquet</code>	columnar, fast, great for big data / cloud
FlatGeobuf	<code>.fgb</code>	streaming / cloud-friendly

```
cols = ["city", "state", "region", "population", "area_km2", "elevation_m", "density", "geometry"]
gdf[cols].to_file(DATA / "brazil.gpkg", layer="capitals", driver="GPKG")
gdf[cols].to_file(OUT / "capitals.geojson", driver="GeoJSON")
gdf[cols].to_parquet(OUT / "capitals.parquet")
```

```
print(gpd.list_layers(DATA / "brazil.gpkg"))
capitals = gpd.read_file(DATA / "brazil.gpkg", layer="capitals")
capitals.head(3)
```

Last executed at 2026-09-25 22:39:06 in 197ms

	name		geometry_type					
0	capitals		Point					
	city	state	region	population	area_km2	elevation_m	density	geometry
0	São Paulo	SP	Sudeste	11451999	1521	760.0	7529.256410	POINT (-46.63 -23.55)
1	Rio de Janeiro	RJ	Sudeste	6211223	1200	5.0	5176.019167	POINT (-43.17 -22.91)
2	Brasília	DF	Centro-Oeste	2817381	5760	1172.0	489.128646	POINT (-47.88 -15.79)

GeoPandas

3.4 Reading and writing vector files

```
# Reading only what you need: attribute filter, bounding box and columns
sudeste = gpd.read_file(DATA / "brazil.gpkg", layer="capitals",
                        where="region = 'Sudeste'", columns=["city", "population"])
print(sudeste)

in_bbox = gpd.read_file(DATA / "brazil.gpkg", layer="capitals", bbox=(-50, -25, -40, -15))
print(in_bbox["city"].tolist())
```

Last executed at 2026-09-25 22:39:06 in 17ms

	city	population	geometry
0	São Paulo	11451999	POINT (-46.63 -23.55)
1	Rio de Janeiro	6211223	POINT (-43.17 -22.91)
2	Belo Horizonte	2315560	POINT (-43.94 -19.92)
3	Vitória	322869	POINT (-40.34 -20.32)

['São Paulo', 'Rio de Janeiro', 'Brasília', 'Belo Horizonte', 'Goiânia', 'Vitória']

Fiona

```
import fiona

with fiona.open(DATA / "brazil.gpkg", layer="capitals") as src:
    print("driver :", src.driver)
    print("crs     :", src.crs)
    print("schema :", src.schema)
    print("n feat :", len(src))
    feat = next(iter(src))
    print("first  :", dict(feat.properties), feat.geometry.coordinates)
```

Last executed at 2026-09-25 22:39:06 in 92ms

```
driver : GPKG
crs     : EPSG:4326
schema : {'properties': {'city': 'str', 'state': 'str', 'region': 'str', 'population': 'int', 'area_km2': 'int', 'elevation_m': 'float', 'density': 'float'}, 'geometry': 'Point'}
n feat : 27
first  : {'city': 'São Paulo', 'state': 'SP', 'region': 'Sudeste', 'population': 11451999, 'area_km2': 1521, 'elevation_m': 760.0, 'density': 7529.25641025641} (-46.63, -23.55)
```

GeoPandas

3.5 crs vs set_crs vs to_crs

Some useful CRS for Brazil:

EPSG	Name	Use
4674	SIRGAS 2000 (geographic)	official Brazilian geodetic datum (IBGE)
31983	SIRGAS 2000 / UTM zone 23S	local/regional work (e.g. São Paulo state)
5880	SIRGAS 2000 / Brazil Polyconic	national maps
ESRI:102033	South America Albers Equal Area Conic	areas at continental scale

```
print(capitals.crs)
print("EPSG code :", capitals.crs.to_epsg())
print("geographic? :", capitals.crs.is_geographic)
print("units :", capitals.crs.axis_info[0].unit_name)
capitals.crs
```

Last executed at 2026-09-25 22:39:06 in 7ms

```
EPSG:4326
EPSG code   : 4326
geographic? : True
units       : degree
<Geographic 2D CRS: EPSG:4326>
Name: WGS 84
Axis Info [ellipsoidal]:
- Lat[north]: Geodetic latitude (degree)
- Lon[east]: Geodetic longitude (degree)
Area of Use:
- name: World.
- bounds: (-180.0, -90.0, 180.0, 90.0)
Datum: World Geodetic System 1984 ensemble
- Ellipsoid: WGS 84
- Prime Meridian: Greenwich
```

```
cap_poly = capitals.to_crs(epsg=5880) # Brazil Polyconic, meters
print(cap_poly.crs.name, "| units:", cap_poly.crs.axis_info[0].unit_name)
print(cap_poly.geometry.head(3))
```

Last executed at 2026-09-25 22:39:06 in 20ms

```
SIRGAS 2000 / Brazil Polyconic | units: metre
0 POINT (5752163.887 7375218.358)
1 POINT (6110051.479 7424570.835)
2 POINT (5655637.917 8244065.991)
Name: geometry, dtype: geometry
```

⚠ set_crs vs to_crs

- **set_crs** only **labels** the data (use it when the CRS is missing but you know what it is). Coordinates do not change.
- **to_crs** **transforms** the coordinates to another CRS.

Using **set_crs** where **to_crs** was needed is one of the most common bugs in GIS!

GeoPandas

3.5 crs vs set_crs vs to_crs

```
wrong = capitals.set_crs(epsg=5880, allow_override=True)  # coordinates are still degrees!  
print("wrong label :", wrong.geometry.iloc[0])  
print("right (to_crs):", cap_poly.geometry.iloc[0])
```

Last executed at 2026-09-25 22:39:06 in 6ms

```
wrong label : POINT (-46.63 -23.55)  
right (to_crs): POINT (5752163.886888129 7375218.358203386)
```

GeoPandas

3.6 Area

```
from shapely.geometry import box

cells = []
for x in range(-75, -30, 5):
    for y in range(-35, 10, 5):
        cells.append({"cell_id": f"{x}_{y}", "geometry": box(x, y, x + 5, y + 5)})
grid = gpd.GeoDataFrame(cells, crs="EPSG:4326")
print(len(grid), "cells")

grid["area_deg2"] = grid.area          # GeoPandas warns: area in a geographic CRS!
grid.head(3)
```

Last executed at 2026-09-25 22:39:06 in 153ms

81 cells

/tmp/ipykernel_6010/3989019928.py:10: UserWarning: Geometry is in a geographic CRS. Results from 'area' are likely incorrect. Use 'GeoSeries.to_crs()' to re-project geometries to a projected CRS before this operation.

grid["area_deg2"] = grid.area # GeoPandas warns: area in a geographic CRS!

	cell_id	geometry	area_deg2
0	-75_-35	POLYGON ((-70 -35, -70 -30, -75 -30, -75 -35, ...	25.0
1	-75_-30	POLYGON ((-70 -30, -70 -25, -75 -25, -75 -30, ...	25.0
2	-75_-25	POLYGON ((-70 -25, -70 -20, -75 -20, -75 -25, ...	25.0

GeoPandas

3.7 Buffer

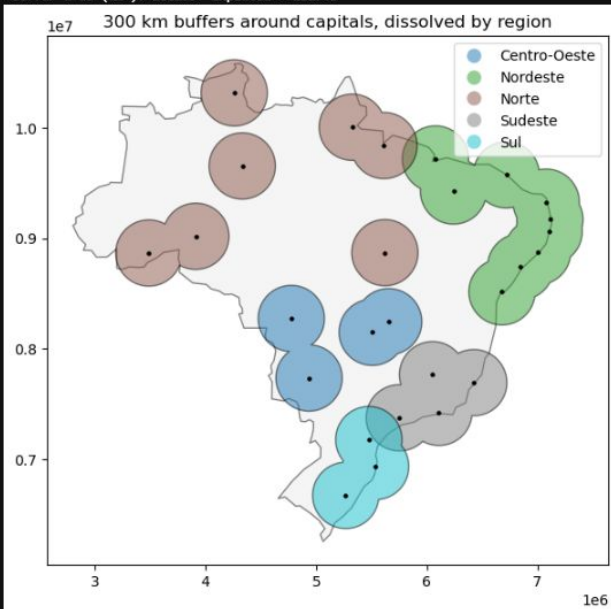
```
buffers = cap_poly.copy()
buffers["geometry"] = cap_poly.buffer(300_000) # 300 km, CRS in meters
print("buffer area (km²):", round(buffers.area.iloc[0] / 1e6), " expected ≈", round(np.pi * 300**2))

# dissolve = the spatial "groupby": merges geometries of the same group
influence = buffers.dissolve(by="region", aggfunc={"population": "sum"}).reset_index()

ax = base_map(plt.subplots(figsize=(7, 7))[1], crs=cap_poly.crs)
influence.plot(ax=ax, column="region", alpha=0.5, edgecolor="k", legend=True)
cap_poly.plot(ax=ax, color="k", markersize=5)
ax.set_title("300 km buffers around capitals, dissolved by region");
```

Last executed at 2026-09-25 22:39:11 in 309ms

buffer area (km²): 282289 expected ≈ 282743



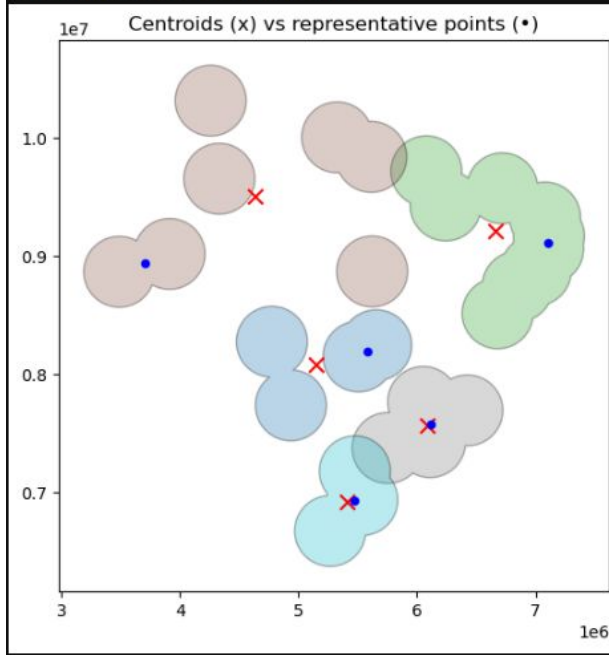
GeoPandas

3.8 Centroids

```
influence_centroids = influence.copy()
influence_centroids["geometry"] = influence.centroid

ax = influence.plot(column="region", alpha=0.3, edgecolor="k", figsize=(6, 6))
influence_centroids.plot(ax=ax, color="red", marker="x", markersize=80)
influence.representative_point().plot(ax=ax, color="blue", markersize=20)
ax.set_title("Centroids (x) vs representative points (•)");
```

Last executed at 2026-09-25 22:39:12 in 222ms



GeoPandas

3.9 Spatial join

```
cells = [
    {"cell_id": "A", "geometry": box(0, 0, 10, 10)},
    {"cell_id": "B", "geometry": box(10, 0, 20, 10)},
    {"cell_id": "C", "geometry": box(0, 10, 10, 20)},
    {"cell_id": "D", "geometry": box(10, 10, 20, 20)},
]
grid = gpd.GeoDataFrame(cells, crs="EPSG:4326")

hotspots = gpd.GeoDataFrame(
    {
        "hotspot_id": range(1, 9),
        "confidence": [85, 92, 71, 88, 65, 95, 79, 90],
    },
    geometry=[
        Point(2, 3), # A
        Point(5, 7), # A
        Point(8, 4), # A
        Point(12, 6), # B
        Point(17, 2), # B
        Point(3, 15), # C
        Point(8, 18), # C
        Point(15, 16), # D
    ],
    crs="EPSG:4326"
)
```

Last executed at 2026-09-25 22:39:12 in 22ms

```
hotspots_grid = gpd.sjoin(
    hotspots,
    grid[["cell_id", "geometry"]],
    predicate="within",
    how="left"
)

print(hotspots_grid[["hotspot_id", "confidence", "cell_id"]])
```

Last executed at 2026-09-25 22:39:12 in 23ms

	hotspot_id	confidence	cell_id
0	1	85	A
1	2	92	A
2	3	71	A
3	4	88	B
4	5	65	B
5	6	95	C
6	7	79	C
7	8	90	D

```
counts = (
    hotspots_grid
    .groupby("cell_id")
    .size()
    .rename("hotspot_count")
)

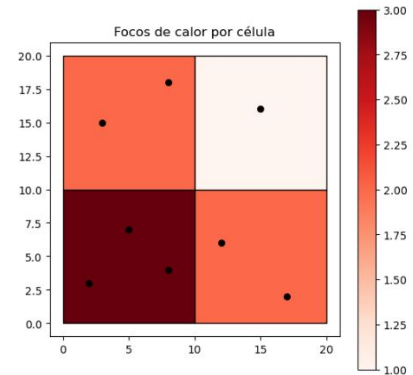
grid = grid.join(counts, on="cell_id")

grid["hotspot_count"] = grid["hotspot_count"].fillna(0).astype(int)

print(grid[["cell_id", "hotspot_count"]])
```

Last executed at 2026-09-25 22:39:12 in 17ms

cell_id	hotspot_count	
0	A	3
1	B	2
2	C	2
3	D	1





—
OGR

OGR

4.1 ogrinfo

4.2 ogr2ogr

OGR

4.1 ogrinfo

```
ogrinfo data/brazil.gpkg
```

```
Last executed at 2026-09-25 22:39:16 in 387ms
```

```
INFO: Open of `data/brazil.gpkg'  
      using driver `GPKG' successful.  
1: capitals (Point)
```

```
# -so = summary only, -al = all layers: CRS, extent, feature count, fields
```

```
ogrinfo -so -al data/brazil.gpkg
```

```
Last executed at 2026-09-25 22:39:16 in 416ms
```

```
INFO: Open of `data/brazil.gpkg'  
      using driver `GPKG' successful.
```

```
Layer name: capitals
```

```
Geometry: Point
```

```
Feature Count: 27
```

```
Extent: (-67.810000, -30.030000) - (-34.860000, 2.820000)
```

```
Layer SRS WKT:
```

```
GEOGCRS["WGS 84",  
  ENSEMBLE["World Geodetic System 1984 ensemble",  
    MEMBER["World Geodetic System 1984 (Transit)"],  
    MEMBER["World Geodetic System 1984 (6730)"],  
    MEMBER["World Geodetic System 1984 (6873)"],  
    MEMBER["World Geodetic System 1984 (61150)"],  
    MEMBER["World Geodetic System 1984 (61674)"],  
    MEMBER["World Geodetic System 1984 (61762)"],  
    MEMBER["World Geodetic System 1984 (62139)"],  
    MEMBER["World Geodetic System 1984 (62296)"],  
    ELLIPSOID["WGS 84",6378137,298.257223563,  
      LENGTHUNIT["metre",1]],  
    ENSEMBLEACCURACY[2.0]],  
  PRIMEM["Greenwich",0,  
    ANGLEUNIT["degree",0.0174532925199433]],  
  CS[ellipsoidal,2],  
    AXIS["geodetic latitude (Lat)",north,  
      ORDER[1],  
        ANGLEUNIT["degree",0.0174532925199433]],  
    AXIS["geodetic longitude (Lon)",east,  
      ORDER[2],  
        ANGLEUNIT["degree",0.0174532925199433]],  
  USAGE[  
    SCOPE["Horizontal component of 3D system."],  
    AREA["World."],  
    BBOX[-90,-180,90,180]],  
  ID["EPSG",4326]]
```

```
Data axis to CRS axis mapping: 2,1
```

```
FID Column = fid
```

```
Geometry Column = geom
```

```
city: String (0.0)
```

```
state: String (0.0)
```

```
region: String (0.0)
```

```
population: Integer64 (0.0)
```

```
area_km2: Integer64 (0.0)
```

```
elevation_m: Real (0.0)
```

```
density: Real (0.0)
```

OGR

4.2 ogr2ogr

```
# Format conversion: GeoPackage -> Shapefile
# (watch the warning: Shapefile column names are limited to 10 characters!)
!ogr2ogr -f "ESRI Shapefile" output/capitals.shp data/brazil.gpkg capitals
!ls output/capitals.*
```

Last executed at 2026-09-25 22:39:18 in 725ms

```
Warning 6: Normalized/launched field name: 'elevation_m' to 'elevation_'
output/capitals.dbf      output/capitals.parquet  output/capitals.shp
output/capitals.geojson  output/capitals.prj      output/capitals.shx
```

```
# ...and back to geographic coordinates (the classic one-liner)
!ogr2ogr -t_srs EPSG:4326 output/capitals_back_4326.shp output/capitals_5880.shp
!ogrinfo -so -al output/capitals_back_4326.shp | grep -E "GEOGCRS|Extent"
```

Last executed at 2026-09-25 22:39:19 in 850ms

```
Extent: (-67.810000, -30.030000) - (-34.860000, 2.820000)
GEOGCRS["WGS 84",
```

```
# Attribute filter (-where), column selection (-select) and GeoJSON output
```

```
!ogr2ogr -f GeoJSON output/big_capitals.geojson data/brazil.gpkg capitals -where "population > 2000000" -select city,population
!head -c 400 output/big_capitals.geojson
```

Last executed at 2026-09-25 22:39:20 in 738ms

```
{
  "type": "FeatureCollection",
  "name": "capitals",
  "crs": { "type": "name", "properties": { "name": "urn:ogc:def:crs:OGC:1.3:CRS84" } },
  "features": [
    { "type": "Feature", "properties": { "city": "São Paulo", "population": 11451999 }, "geometry": { "type": "Point", "coordinates": [ -46.63, -23.55 ] } },
    { "type": "Feature", "properties": { "city": "Rio de Janeiro", "population": 6211223 }, "geom
```



—

Raster data

Common Raster Data Formats



Format	Common use	Advantages	Limitations	Introduced
GeoTIFF / COG*	GIS, Earth observation, DEMs	Georeferencing embedded; broad software support; compression; COG enables efficient remote access	Storage and I/O depend on compression and tiling configuration; limited multidimensional support; COG requires specific internal organization	1995 / 2023*
JPEG 2000	Satellite imagery, compressed imagery	Lossless or lossy compression; multiresolution access; high compression efficiency	Slower encoding/decoding; less universal GIS support than GeoTIFF	2001
NetCDF	Climate, oceanography, time series, data cubes	Multidimensional arrays; rich metadata; well suited to space-time data	More complex structure; performance depends on chunking and organization	1989
HDF5	Satellite products, scientific multidimensional data	Hierarchical structure; compression; large multidimensional datasets	Complex structure; georeferencing conventions vary among products	1998
GRIB2	Meteorological and weather forecast data, numerical weather prediction, atmospheric and oceanographic fields	Compact binary representation; efficient data transmission and storage; supports many meteorological parameters, grids and projections; standardized by WMO	Primarily designed for meteorological data; less flexible for general multidimensional scientific datasets; relatively complex structure; metadata and parameter tables can be difficult to interpret	2000
Zarr	Large data cubes, distributed and remote array processing	Chunked N-D arrays; parallel access; efficient subset reading	Many-file/object structure; geospatial conventions still evolving	2016
GeoZarr	Cloud-native geospatial data cubes, Earth observation, multidimensional raster data, large-scale remote sensing	Chunked multidimensional arrays; efficient parallel and remote access; supports geospatial metadata and CRS; well suited to cloud/object storage; enables efficient subset and multiscale access	Geospatial conventions are still being standardized; ecosystem and software support are less mature than GeoTIFF; can involve many chunks/objects and require careful chunking strategy	2024



GDAL

GDAL

GDAL is a translator library for raster and vector geospatial data formats that is released under an MIT style Open Source License by the **Open Source Geospatial Foundation**

• Raster drivers

- AAIGrid -- Arc/Info ASCII Grid
- ACE2 -- ACE2
- ADRG -- ADRG/ARC Digitized Raster Graphics (.gen/.thf)
- AIG -- Arc/Info Binary Grid
- AIRSAR -- AIRSAR Polarimetric Format
- AVIF -- AV1 Image File Format
- BAG -- Bathymetry Attributed Grid
- BASISU -- Basis Universal
- BMP -- Microsoft Windows Device Independent Bitmap
- BSB -- Maptech/NOAA BSB Nautical Chart Format
- BT -- VTP .bt Binary Terrain Format
- BYN - Natural Resources Canada's Geoid file format (.byn)
- CAD -- AutoCAD DWG raster layer

- GTiff -- GeoTIFF File Format
- GXF -- Grid eXchange File
- HDF4 -- Hierarchical Data Format Release 4 (HDF4)
- HDF5 -- Hierarchical Data Format Release 5 (HDF5)
- JP2OpenJPEG -- JPEG2000 driver based on OpenJPEG library
- JPEG -- JPEG JFIF File Format
- HDF4 -- Hierarchical Data Format Release 4 (HDF4)
- HDF5 -- Hierarchical Data Format Release 5 (HDF5)
- GRIB -- WMO General Regularly-distributed Information in Binary
- MEM -- In Memory datasets
- NetCDF: Network Common Data Form
- Zarr

• Vector drivers

- ADBC -- Arrow Database Connectivity
- ADBC BigQuery -- Google BigQuery through Arrow Database Connectivity
- Artificial intelligence powered vector driver
- AmigoCloud
- (Geo)Arrow IPC File Format / Stream
- Arc/Info Binary Coverage
- Arc/Info E00 (ASCII) Coverage
- CAD -- AutoCAD DWG
- Carto
- Comma Separated Value (.csv)
- CSW - OGC CSW (Catalog Service for the Web)
- Microstation DGN
- Microstation DGN v8
- AutoCAD DWG
- AutoCAD DXF
- Google Earth Engine Data API
- EDIGEO
- Elasticsearch: Geographically Encoded Objects for Elasticsearch
- GeoJSON
- GPKG -- GeoPackage vector
- ESRI Shapefile / DBF
- (Geo)Parquet
- VRT -- OGR Virtual Format

GDAL

```
sudo apt-get update && sudo apt-get install -y g++ gdal-bin libgdal-dev  
pip install gdal  
conda install gdal
```

Programs:

gdalinfo: raster metadata -stats -hist

gdal_translate: Converts raster data between different formats

gdalwarp: Image reprojection and warping utility

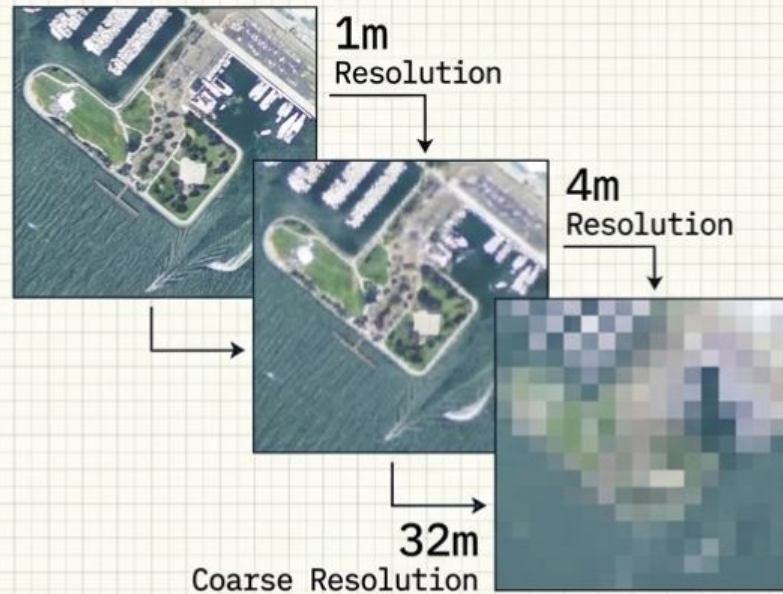
gdal_merge:

gdalcalc: raster calculator

Overviews & Blocks

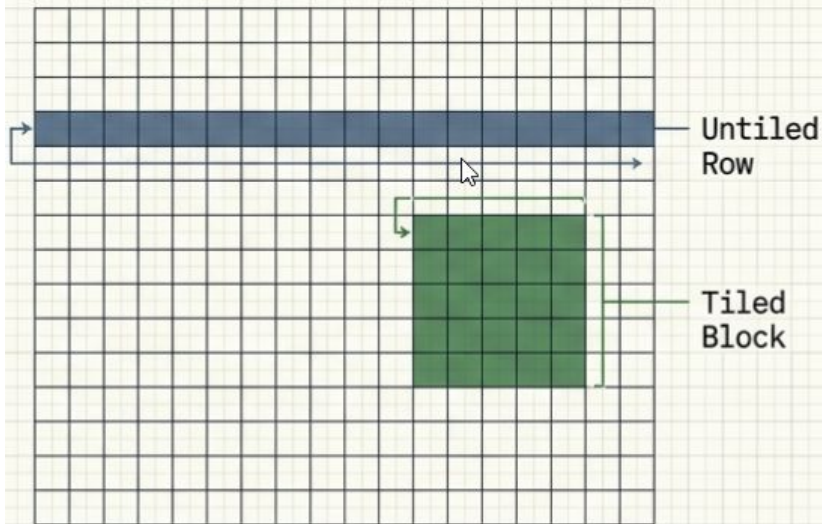
Overviews & Pyramids

Mechanism for display speed. Reduced-resolution layers that draw instantly when zoomed out. Build decreasing resolutions until the coarsest layer is 256 pixels or smaller.



Disk Blocks

Mechanism for Read/Write efficiency. Represents how pixels are physically clustered on the hard drive. Tiled GeoTIFFs use square chunks (e.g., 256x256), while untiled formats read full rows.



Overviews: 2700x2250, 1350x1125, 675x562, 337x281

STATISTICS VALID PERCENT=100

Warning 1: Unable to save auxiliary information in <https://data.inpe.br/bdc/data/topodata/v001/23S/465/23S465SN.tif.aux.xml>.

GDAL

5.2 gdal_translate

```
# Compression NONE
!gdal_translate -q -co COMPRESS=NONE -co PREDICTOR=3 -co TILED=YES "https://data.inpe.br/bdc/data/topodata/v001/23S/46S/23S46S46S.tif" output/dem_deflate.tif

# Compression and tiling (PREDICTOR=3 = floating-point predictor; use 2 for integer data)
!gdal_translate -q -co COMPRESS=DEFLATE -co PREDICTOR=3 -co TILED=YES data/dem.tif output/dem_deflate.tif

# Cloud Optimized GeoTIFF (COG): tiled + overviews + HTTP range-request friendly
!gdal_translate -q -of COG -co COMPRESS=DEFLATE -co PREDICTOR=3 data/dem.tif output/dem_cog.tif

!ls -lh data/dem.tif output/dem_deflate.tif output/dem_cog.tif
```

```
# Subset by pixel window (-srcwin xoff yoff xsize ysize) or by coordinates (-projwin ulx uly lrx lry)
!gdal_translate -q -srcwin 0 0 200 150 data/dem.tif output/dem_subset_px.tif
!gdal_translate -q -projwin 400000 7445000 410000 7438000 data/dem.tif output/dem_subset_xy.tif

# Resample to 25% of the size
!gdal_translate -q -outsize 25% 25% -r average data/dem.tif output/dem_small.tif

# Select / reorder bands: false color composite (NIR, Red, Green)
!gdal_translate -q -b 4 -b 3 -b 2 data/multispectral.tif output/false_color.tif

# To an 8-bit PNG (scaled 500..2000 m -> 0..255), e.g. for a quick figure or web
!gdal_translate -q -of PNG -ot Byte -scale 500 2000 0 255 data/dem.tif output/dem.png
```

GDAL

5.3 gdalwarp

```
# Reproject to geographic coordinates
!gdalwarp -q -overwrite -t_srs EPSG:4326 -r bilinear data/dem.tif output/dem_4326.tif
!gdalinfo output/dem_4326.tif | grep -E "Size is|Pixel Size|GEOGCRS"
```

Last executed at 2026-09-25 22:40:04 in 2.68s

```
Size is 5400, 4500
GEOGCRS["WGS 84",
Pixel Size = (0.000277778193079,-0.000277778193079)
```

```
# Change resolution (30 m -> 90 m); choose the resampling method carefully:
# nearest -> categorical data (land cover classes); bilinear/cubic/average -> continuous data
!gdalwarp -q -overwrite -tr 90 90 -r average data/dem.tif output/dem_90m.tif
!gdalinfo output/dem_90m.tif | grep -E "Size is|Pixel Size"
```

Execution started at 2026-09-25 22:40:04

1h 3m

```
# Clip by a polygon (cutline)
!gdalwarp -q -overwrite -cutline data/aoi.gpkg -crop_to_cutline -dstnodata -9999 data/dem.tif output/dem_aoi.tif
!gdalinfo output/dem_aoi.tif | grep -E "Size is"
```

GDAL

5.4 gdal_merge

```
!gdal_merge.py -q -o output/dem_merged.tif -n -9999 -a_nodata -9999 data/dem_tile_left.tif data/dem_tile_right.tif
!gdalinfo output/dem_merged.tif | grep -E "Size is|Origin"

!gdalbuildvrt -q output/dem_mosaic.vrt data/dem_tile_left.tif data/dem_tile_right.tif
!head -12 output/dem_mosaic.vrt
```

5.5 gdal_calc

```
# NDVI = (NIR - Red) / (NIR + Red) -> NIR is band 4, Red is band 3
!gdal_calc.py --quiet --overwrite -A data/multispectral.tif --A_band=4 -B data/multispectral.tif --B_band=3 \
  --outfile=output/ndvi.tif --type=Float32 --NoDataValue=-9999 \
  --calc="(A.astype(float) - B) / (A.astype(float) + B)"

# A binary mask: areas above 1000 m
!gdal_calc.py --quiet --overwrite -A data/dem.tif --outfile=output/dem_above_1000.tif --type=Byte \
  --calc="A > 1000"

!gdalinfo -stats output/ndvi.tif | grep -E "STATISTICS_(MIN|MAX|MEAN)"
```

GDAL in Python

```
from osgeo import gdal
import sys

gdal.UseExceptions()

ds = gdal.Open(sys.argv[1])
print(f"Width: {ds.RasterXSize}")
print(f"Height: {ds.RasterYSize}")
print(f"Number of bands: {ds.RasterCount}")

array = ds.GetRasterBand(1).ReadAsArray()
print(array)
```

```
ds = gdal.Open("data/dem.tif")
print("Driver      :", ds.GetDriver().ShortName)
print("Size       :", ds.RasterXSize, "x", ds.RasterYSize, "x", ds.RasterCount)
gt = ds.GetGeoTransform()
print("GeoTransform:", gt)

srs = osr.SpatialReference(wkt=ds.GetProjection())
print("CRS         :", srs.GetName(), "| EPSG:", srs.GetAuthorityCode(None))

band = ds.GetRasterBand(1)
print("Data type  :", gdal.GetDataTypeName(band.DataType))
print("NoData     :", band.GetNoDataValue())
print("Stats      :", band.ComputeStatistics(False)) # [min, max, mean, std], ignoring nodata

# Pixel -> map coordinates
col, row = 100, 50
x = gt[0] + col * gt[1] + row * gt[2]
y = gt[3] + col * gt[4] + row * gt[5]
print(f"pixel (col={col}, row={row}) -> x={x}, y={y}")
```

Last executed at 2026-09-25 23:48:15 in 383ms

```
Driver      : GTiff
Size        : 5400 x 4500 x 1
GeoTransform: (-46.5, 0.000277777784814818, 0.0, -23.0, 0.0, -0.00027777777777778)
CRS         : WGS 84 | EPSG: 4326
Data type   : Float32
NoData      : -9999.0
Stats       : [0.0, 381.64013671875, 16.62099006740942, 16.89175617950065]
pixel (col=100, row=50) -> x=-46.47222215185185, y=-23.01388888888889
```

GDAL in Python

```
arr = band.ReadAsArray() # the full band as a NumPy array
print(arr.shape, arr.dtype)
print("naive mean (includes -9999!):", arr.mean())
print("valid mean      :", arr[arr != band.GetNoDataValue()].mean())
# Read only a window: ReadAsArray(xoff, yoff, xsize, ysize)
win = band.ReadAsArray(100, 50, 5, 4)
print(win)
```

Last executed at 2026-09-25 23:48:16 in 303ms

```
(4500, 5400) float32
naive mean (includes -9999!): 16.620987
valid mean      : 16.620987
[[16.205795  13.368015  9.420291  5.813799  3.314    ]
 [18.910292  17.078543  14.789304  8.642873  5.5171676]
 [19.515333  18.251778  17.596546  14.992447  12.8039   ]
 [20.530474  20.76429  19.691832  18.745113  16.521694 ]]
```

```
# Classify slope into 3 classes: 0 = flat (< 8°), 1 = moderate (8-20°), 2 = steep (> 20°)
slope_ds = gdal.Open("https://data.inpe.br/bdc/data/topodata/v001/235/465/235465SN.tif")
slope = slope_ds.GetRasterBand(1).ReadAsArray()
slope_nodata = slope_ds.GetRasterBand(1).GetNoDataValue()
slope_cls = np digitize(slope, [8, 20]).astype(np.uint8)
slope_cls[slope == slope_nodata] = 255
```

```
driver = gdal.GetDriverByName("GTiff")
out = driver.Create("output/slope_classes.tif", slope_ds.RasterXSize, slope_ds.RasterYSize, 1,
                    gdal.GDT_Byte, options=["COMPRESS=DEFLATE"])
out.SetGeoTransform(slope_ds.GetGeoTransform())
out.SetProjection(slope_ds.GetProjection())
out_band = out.GetRasterBand(1)
out_band.WriteArray(slope_cls)
out_band.SetNoDataValue(255)
out = None # close -> flush to disk
slope_ds = None
```

```
print(np.unique(slope_cls, return_counts=True))
```

Last executed at 2026-09-25 23:51:20 in 8.28s

```
(array([0, 1, 2], dtype=uint8), array([11249459, 4337047, 8713494]))
```



Rasterio

Rasterio

7.1 Opening a dataset and reading its metadata

```
import rasterio

print("rasterio", rasterio.__version__, "| GDAL", rasterio.__gdal_version__)

img_url = "https://data.inpe.br/bdc/data/S2_L2A/v001/20/LP/2026/09/S2B_TCI_20260924T143739_N0513_R096_T20LLP_20260924T18120"
with rasterio.open(img_url) as src:
    print("width :", src.width)
    print("height :", src.height)
    print("count :", src.count)          # number of bands
    print("crs :", src.crs)
    print("transform:\n", src.transform)
    print("bounds :", src.bounds)
    print("dtypes :", src.dtypes)
    print("nodata :", src.nodata)
    print("res :", src.res)
    print("profile :", src.profile)
```

Last executed at 2026-09-25 23:53:37 in 3.45s

```
rasterio 1.4.3 | GDAL 3.9.3
width      : 10980
height     : 10980
count      : 3
crs        : EPSG:32720
transform:
| 10.00, 0.00, 300000.00|
| 0.00,-10.00, 8900020.00|
| 0.00, 0.00, 1.00|
bounds     : BoundingBox(left=300000.0, bottom=8790220.0, right=409800.0, top=8900020.0)
dtypes      : ('uint8', 'uint8', 'uint8')
nodata     : None
res        : (10.0, 10.0)
profile    : {'driver': 'Gtiff', 'dtype': 'uint8', 'nodata': None, 'width': 10980, 'height': 10980, 'count': 3, 'crs': CRS.from
            _wkt('PROJCS["WGS 84 / UTM zone 20S",GEOGCS["WGS 84",DATUM["WGS 1984",SPHEROID["WGS 84",6378137,298.257223563,AUTHORITY["EPS
            G","7030"]],AUTHORITY["EPSG","6326"]],PRIMEM["Greenwich",0,AUTHORITY["EPSG","8901"]],UNIT["degree",0.0174532925199433,AUTHORI
            TY["EPSG","9122"]],AUTHORITY["EPSG","4326"]],PROJECTION["Transverse_Mercator"],PARAMETER["latitude_of_origin",0],PARAMETER["c
            entral_meridian",-63],PARAMETER["scale_factor",0.9996],PARAMETER["false_easting",500000],PARAMETER["false_northing",1000000
            0],UNIT["metre",1,AUTHORITY["EPSG","9001"]],AXIS["Easting",EAST],AXIS["Northing",NORTH],AUTHORITY["EPSG","32720"]')}, 'transf
            orm': Affine(10.0, 0.0, 300000.0,
            0.0, -10.0, 8900020.0), 'blocksize': 512, 'blocksize': 512, 'tiled': True, 'compress': 'deflate', 'interleave': 'pix
            el'}
```

Rasterio

7.2 read

```
band1 = rasterio.open(img_url).read(1)          # (rows, cols) -- bands are 1-based!
print(band1.shape, band1.shape, band1.dtype)

with rasterio.open(img_url) as ms_src:
    green, red = ms_src.read([2, 3]).astype(np.float32)
    print("multispectral:", ms_src.count, "bands", ms_src.dtypes, "| band descriptions:", ms_src.descriptions)
```

Last executed at 2026-09-25 23:55:46 in 25.71s

(10980, 10980) (10980, 10980) uint8

multispectral: 3 bands ('uint8', 'uint8', 'uint8') | band descriptions: (None, None, None)

band1

Last executed at 2026-09-25 23:55:46 in 8ms

```
array([[ 99,  99, 102, ...,  0,  0,  0],
       [100,  98, 102, ...,  0,  0,  0],
       [100, 100, 100, ...,  0,  0,  0],
       ...,
       [ 23,  20,  24, ...,  0,  0,  0],
       [ 20,  21,  26, ...,  0,  0,  0],
       [ 20,  23,  26, ...,  0,  0,  0]], dtype=uint8)
```

Rasterio

7.3 masked

```
img_url2 = "https://data.inpe.br/bdc/data/s2-16d/v2/001/014/2026/08/13/S2-16D_V2_001014_20260813_001.tif"
with rasterio.open(img_url2) as src:
    blue = src.read(1)
```

Last executed at 2026-09-25 23:56:45 in 2.22s

```
with rasterio.open(img_url2) as src:
    blue_m = src.read(1, masked=True)
    print(type(blue_m))
    print("masked pixels:", blue_m.mask.sum())
    print(f"mean without mask: {blue_m.mean():9.2f}")
    print(f"mean with mask : {blue_m.mean():9.2f}")
    print(f"min / max : {blue_m.min():.1f} / {blue_m.max():.1f}")
```

```
# The dataset-level mask (255 = valid, 0 = nodata)
print(np.unique(src.dataset_mask(), return_counts=True))
```

Last executed at 2026-09-25 23:56:56 in 10.75s

```
<class 'numpy.ma.core.MaskedArray'>
masked pixels: 85614915
mean without mask: 48.62
mean with mask : 209.35
min / max : 18.0 / 9520.0
(array([ 0, 255], dtype=uint8), array([85614915, 25898685]))
```

```
with rasterio.open(img_url) as src:
    tci_blue = src.read(1)
```

Last executed at 2026-09-25 23:57:08 in 12.26s

```
with rasterio.open(img_url) as src:
    tci_blue_m = src.read(1, masked=True)
    print(type(blue_m))
    print("masked pixels:", tci_blue_m.mask.sum())
    print(f"mean without mask: {tci_blue_m.mean():9.2f}")
    print(f"mean with mask : {tci_blue_m.mean():9.2f}")
    print(f"min / max : {tci_blue_m.min():.1f} / {tci_blue_m.max():.1f}")
```

```
# The dataset-level mask (255 = valid, 0 = nodata)
print(np.unique(src.dataset_mask(), return_counts=True))
```

Last executed at 2026-09-25 23:57:24 in 15.48s

```
<class 'numpy.ma.core.MaskedArray'>
masked pixels: 0
mean without mask: 35.94
mean with mask : 35.94
min / max : 0.0 / 255.0
(array([255], dtype=uint8), array([120560400]))
```

Rasterio

7.4 Window

```
from rasterio.windows import Window, from_bounds

src = rasterio.open(img_url)

win = Window(col_off=100, row_off=50, width=200, height=150)
sub = src.read(1, window=win, masked=True)
print("window shape:", sub.shape)

win_transform = src.window_transform(win)  # the georeferencing of the window
print("window origin:", win_transform.c, win_transform.f)

# A window from map coordinates
win2 = from_bounds(401000, 7438000, 407000, 7444000, transform=src.transform)
print(win2)

# Save the window as a new GeoTIFF: copy the profile and update what changed
profile = src.profile.copy()
profile.update(width=win.width, height=win.height, transform=win_transform, compress="deflate")
with rasterio.open("output/dem_window.tif", "w", **profile) as dst:
    dst.write(src.read(1, window=win), 1)

Last executed at 2026-09-25 23:58:16 in 281ms

window shape: (150, 200)
window origin: 301000.0 8899520.0
Window(col_off=10100.0, row_off=145602.0, width=600.0, height=600.0)

# Processing block by block (windows follow the internal tiling/stripping of the file)
maxima = [src.read(1, window=w, masked=True).max() for _, w in src.block_windows(1)]
print(len(maxima), "blocks | global max:", max(m for m in maxima if m is not np.ma.masked))

# Or regular tiles, e.g. 256 x 256 chips for a CNN
chips = [Window(c, r, 256, 256) for r in range(0, src.height - 255, 256) for c in range(0, src.width - 255, 256)]
print(len(chips), "full 256x256 chips")

Last executed at 2026-09-25 23:58:31 in 14.50s

484 blocks | global max: 255
1764 full 256x256 chips
```

All together

```
red_url = "https://data.inpe.br/bdc/data/S2_L2A/v001/22/J/BN/2025/07/S2B_804_202"
nir_url = "https://data.inpe.br/bdc/data/S2_L2A/v001/22/J/BN/2025/07/S2B_808_202"

with rasterio.open(red_url) as red_src:
    red = red_src.read(1, masked=True).astype("float32")
    profile = red_src.profile
    raster_crs = red_src.crs

with rasterio.open(nir_url) as nir_src:
    nir = nir_src.read(1, masked=True).astype("float32")

ndvi = (nir - red) / (nir + red)

profile.update(dtype="float32", nodata=-9999, count=1)
with rasterio.open("data/ndvi.tif", "w", **profile) as dst:
    dst.write(ndvi.filled(-9999).astype("float32"), 1)

# from rasterio.io import MemoryFile
# with MemoryFile() as memfile:
#     with memfile.open(**profile) as ndvi_src:
#         ndvi_src.write(ndvi.filled(-9999).astype("float32"), 1)
#         ndvi_clip, transform = mask(ndvi_src, geoms, crop=True, nodata=-9999)
```

```
gleba_file = 'https://github.com/brazil-data-cube/code-gallery/raw/main'
ref_bacen = 515791395
glebas = gpd.read_file(gleba_file)
gleba = glebas[glebas['REF_BACEN'] == ref_bacen]
gleba_crsraster = gleba.to_crs(raster_crs)
geom = gleba_crsraster.geometry
bounds = geom.bounds

print(f"Geometria: {geom}")
print()
print(f"Tipo da geometria: {geom.geom_type}")
print()
print(f"Bounds: {bounds}")
```

Last executed at 2026-09-25 23:59:33 in 5.17s

Geometria: 37 POLYGON ((281753.027 6796416.352, 281460.486 6...
Name: geometry, dtype: geometry

Tipo da geometria: 37 Polygon
dtype: object

Bounds:	minx	miny	maxx	maxy
37	281348.923766	6.796344e+06	283175.539446	6.797512e+06

All together

```
from rasterio.mask import mask

with rasterio.open("data/ndvi.tif") as src:
    ndvi_clip, transform = mask(src, geom, crop=True, nodata=-9999)
    print("CRS do raster:", src.crs)
    print("CRS da gleba:", gleba.crs)
    print("CRS da gleba:", gleba_crsraster.crs)

    print(src.bounds)
    print(gleba_crsraster.bounds)
```

```
values = ndvi_clip[0]
values = values[values != -9999]

print("N pixels:", len(values))
print(f"Mean NDVI: {values.mean():.4f}")
```

Last executed at 2026-09-26 00:02:59 in 36ms

CRS do raster: EPSG:32722

CRS da gleba: EPSG:4674

CRS da gleba: PROJCS["WGS 84 / UTM zone 22S",GEOGCS["WGS 84",DATUM["WGS_1984",SPHEROID["WGS 84",6378137,298.257223563,AUTHORITY["EPSG","7030"]],AUTHORITY["EPSG","6326"]],PRIMEM["Greenwich",0,AUTHORITY["EPSG","8901"]],UNIT["degree",0.0174532925199433,AUTHORITY["EPSG","9122"]],AUTHORITY["EPSG","4326"]],PROJECTION["Transverse_Mercator"],PARAMETER["latitude_of_origin",0],PARAMETER["central_meridian",-51],PARAMETER["scale_factor",0.9996],PARAMETER["false_easting",500000],PARAMETER["false_northing",1000000],UNIT["metre",1,AUTHORITY["EPSG","9001"]],AXIS["Easting",EAST],AXIS["Northing",NORTH],AUTHORITY["EPSG","32722"]]

	minx	miny	maxx	maxy
37	281348.923766	6.796344e+06	283175.539446	6.797512e+06

N pixels: 11242

Mean NDVI: 0.4651

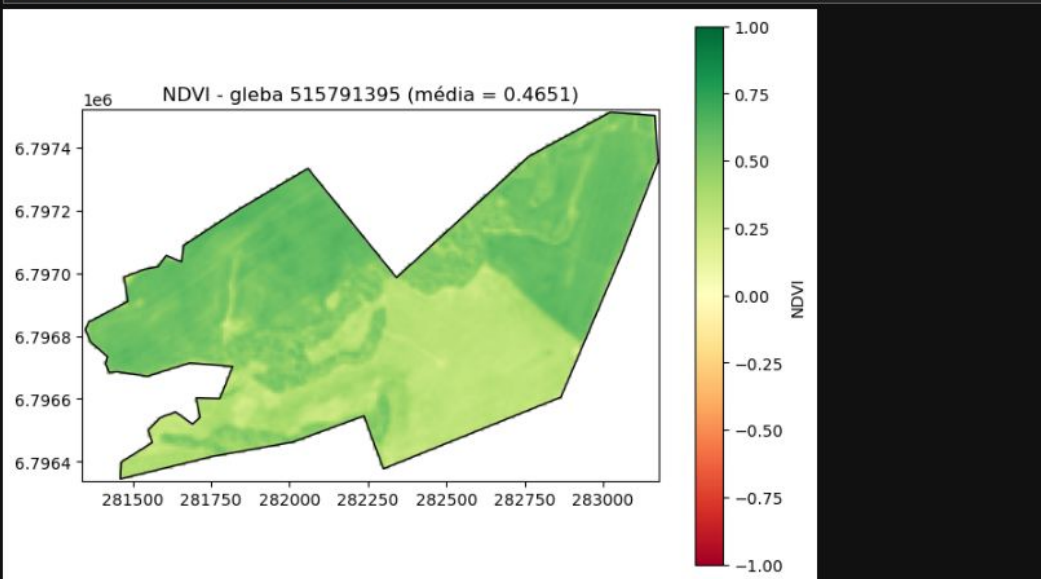
All together

```
from rasterio.plot import show

ndvi_plot = np.ma.masked_equal(ndvi_clip[0], -9999)

fig, ax = plt.subplots(figsize=(8, 6))
img = show(ndvi_plot, transform=transform, ax=ax, cmap="RdYlGn", vmin=-1, vmax=1)
gleba_crsraster.boundary.plot(ax=ax, color="black", linewidth=1)
fig.colorbar(img.get_images()[0], ax=ax, label="NDVI")
ax.set_title(f"NDVI - gleba {ref_bacen} (média = {ndvi_mean:.4f})" if False else f"NDVI - gleba {ref_bacen} (média = {np.mean(ndvi_clip[0]):.4f})")
plt.show()
```

Last executed at 2026-09-26 00:04:20 in 617ms



Task



1

Read Numpy article

natureView

[Explore content](#) ▾ [About the journal](#) ▾ [Publish with us](#) ▾

[nature](#) > [review articles](#) > [article](#)

Review Article | [Open access](#) | Published: 16 September 2020

Array programming with NumPy

[Charles R. Harris](#), [K. Jarrod Millman](#) ✉, [Stéfan J. van der Walt](#) ✉, [Ralf Gommers](#) ✉, [Pauli Virtanen](#), [David Cournapeau](#), [Eric Wieser](#), [Julian Taylor](#), [Sebastian Berg](#), [Nathaniel J. Smith](#), [Robert Kern](#), [Matti Picus](#), [Stephan Hoyer](#), [Marten H. van Kerkwijk](#), [Matthew Brett](#), [Allan Haldane](#), [Jaime Fernández del Río](#), [Mark Wiebe](#), [Pearu Peterson](#), [Pierre Gérard-Marchant](#), [Kevin Sheppard](#), [Tyler Reddy](#), [Warren Weckesser](#), [Hameer Abbasi](#), ... [Travis E. Oliphant](#) [+ Show authors](#)

[Nature](#) **585**, 357–362 (2020) | [Cite this article](#)

[Save article](#)

710k Accesses | **29k** Citations | **1645** Altmetric | [Metrics](#)

Abstract

Array programming provides a powerful, compact and expressive syntax for accessing, manipulating and operating on data in vectors, matrices and higher-dimensional arrays. NumPy

Task



2

Practice the "all together" section using R

Geospatial file formats and packages

Rennan Marujo
Édipo Cremon

